

TimeBox Testing Lab: Building a Small Fault-Injection Project for Exploring Test Flakiness

Hamiz Siddiqui

April 2026

Abstract

This project started as a simple task manager with tests, but the final version became a small fault-injection lab focused on demonstrating how automated tests behave under unstable conditions. The application was built with FastAPI, SQLite, Jinja templates, Pytest, and Playwright. Instead of only checking the normal CRUD path, the system can turn on specific fault profiles such as duplicate task creation, case-sensitive search regression, unstable sorting, stale dashboard counts, slow UI behavior, and midnight boundary issues. During development, the project changed in a few useful ways: fault controls moved from API-only calls into the website itself, the state of those faults became session-based instead of global, editable tasks were added, the user interface was redesigned, and the demo screenshot flow was fixed so it captured the whole page. In the final version, the project works as a self-contained testing demo with passing API and UI tests and a visible browser walkthrough that is much easier to present in class.

1. Introduction

Test flakiness is frustrating because it makes a test suite feel unreliable even when the code is mostly fine. A flaky test may pass once, fail the next time, and then pass again without a real code change. That usually means the test is sensitive to timing, ordering, stale state, or some hidden assumption about the environment. For a class project, I wanted something smaller and more concrete than a giant production-style study, but still close to the real idea of unstable testing conditions.

The result is *TimeBox Testing Lab*, a web app that looks like a task manager on the surface but is really meant to act as a testing playground. The main idea is simple: keep the application small enough to understand quickly, then inject a few controlled bugs that are easy to trigger, observe, and test. This approach gave me a project that is easier to explain than a broad research platform, while still showing the main lesson that “good” tests can behave badly when the software under them stops being stable.

I also intentionally pushed the project away from feeling like a copy of another student’s proposal. Instead of building a huge matrix of operating systems, clock settings, and deployment environments, I focused on an interactive lab where the user can turn faults on inside the interface and immediately see how the app and tests react. That made the final result more original, more demo-friendly, and honestly more manageable within the scope of this assignment.

2. Technical Details

The application uses FastAPI for the backend, SQLite for persistence, server-rendered Jinja templates for the interface, and Playwright plus Pytest for automation. The database has just enough structure to support the demo: users can register and sign in, then create, edit, complete, search, filter, sort, and delete tasks. This basic task flow matters because it gives the faults something realistic to break.

The fault model is the core of the project. Six named profiles are supported: `slow_ui`, `duplicate_create_bug`, `unstable_sort_bug`, `search_bug`, `dashboard_cache_bug`, and `midnight_boundary`. Each one targets a different kind of failure. For example, duplicate creation simulates bad form handling, unstable sort removes the stable secondary ordering key, search regression makes matching case-sensitive, and midnight boundary shifts the effective current

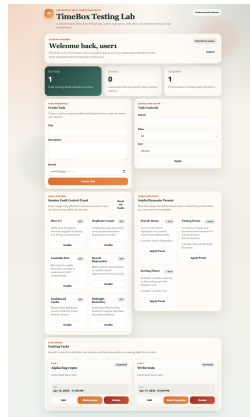


Figure 1: Full-page demo capture after the screenshot flow was fixed. The report version uses the same dashboard view shown during the visible browser demo.

time to expose due-date edge cases. These are not random fake errors. They are small examples of bug classes that show up often enough in real systems to be worth demonstrating.

One of the biggest design changes happened after I looked at how awkward the project felt when every fault had to be turned on through an API call. That worked for automation, but it was not a good presentation experience. I changed the app so the dashboard includes a “Session Fault Control Panel” with on/off buttons for each bug and one-click scenario presets for search, timing, and sorting demos. The API still exists for tests, but the interface is now the primary way to interact with the lab. This made the project feel more like a real testing tool and less like a hidden backend trick.

Another important fix was fault scope. At first, the fault state was global to the whole app. That meant if one user turned on a bug, everybody saw it. For a demo system, that is messy. I changed the implementation to store the fault profile in the session instead. In practice, this means one browser session can run a buggy scenario while another session stays clean. That change also made the project easier to reason about because the test setup became more isolated.

The app itself also improved over time. I added task editing so the project did not stop at create/delete actions. The dashboard was redesigned to look more intentional, with a stronger landing section, a cleaner task board, and a more obvious split between the task workspace and the fault controls. I also fixed a small but annoying usability problem where pressing buttons kept sending the page back to the top. A lightweight scroll-restore script now keeps the user near the place where the action happened. This was not a flashy bug, but it mattered because it made the app much less irritating to use during the demo.

Problems we ran into during development.

Some of the most useful work on the project came from issues that were not obvious at the start. The first version technically worked, but it felt more like a plain app with a few tests attached than a real testing lab. That gap showed up most clearly during demo runs. API-only controls were awkward, the visual design was flat, and the app did not support editing existing tasks, which made the task flow feel thinner than it should have. Fixing those things took time, but they mattered because the assignment is not just about backend correctness. It is also about presenting a coherent system that clearly communicates what is being tested and why.

Another small but real problem was friction between the interface and the demo artifacts. After button clicks, the dashboard would jump back to the top of the page, which made the interaction feel rough. The first round of saved screenshots had the opposite issue: they were taken from the middle of the page, so they missed the main header and did not show the whole app. Neither of those issues sounds dramatic, but both made the project look less complete than it really was. Once they were fixed, the lab felt much more polished and easier to explain.

A final technical issue came from the presentation pipeline itself. The first round of saved screenshots only captured the current viewport, which often meant the image showed the middle of the page instead of the full dashboard. Since this project is partly about communication and not only code, that was a real problem. The demo runner was updated to wait for the page to settle, scroll back to the top, and save a full-page screenshot. That gave me cleaner figures and made the final output feel much more complete.

3. Experimental Results

The experimental side of this project is intentionally small but concrete. Rather than reporting a large statistical study, I treated the app as a repeatable testing sandbox and checked whether each fault profile produced the behavior it was supposed to expose. The final automated suite passed with **9 API tests** and **4 Playwright UI tests**. Those tests cover normal task creation, session isolation of fault profiles, preset behavior, duplicate submission visibility, search failure under the regression profile, task editing, and the basic end-to-end happy path.

Table 1 summarizes the most useful scenarios. The point is not that every scenario is wildly complex. The point is that each one creates an understandable failure mode that can be observed in both the interface and the automated tests.

Scenario	Faults Enabled	Observed Result
Happy path	none	Tasks can be created, searched, edited, completed, and deleted normally.
Duplicate create	<code>duplicate_create_bug</code>	One submit action produces two task rows, which is easy to verify in UI and tests.
Search demo	<code>search_bug</code>	Searching for “alpha” fails to return “Alpha Task,” showing a regression that looks small but breaks expected behavior.
Timing demo	<code>slow_ui + midnight_boundary</code>	The interface visibly slows down and time-sensitive status logic becomes easier to challenge around due-date boundaries.
Sorting demo	<code>unstable_sort_bug</code>	Task ordering becomes less predictable because the stable tie-break behavior is removed.

Table 1: Representative scenarios used in the final project demo.

From a testing point of view, the strongest result was not the raw number of tests. It was the fact that the project became easier to demonstrate and easier to explain after the tooling matched the goal better. Session-based faults made tests more isolated. UI toggles and presets made the demos more understandable. Editable tasks made the app feel less fake. The screenshot fix made the final report artifacts usable. In other words, the project got better not only by adding more features, but by removing friction between the code, the tests, and the way the project is presented.

There were still limitations. This is not a full production flakiness study. It does not sweep across multiple operating systems, randomized seeds, or cloud environments, and it does not claim a real flake rate for a large system. The tradeoff was intentional. I chose depth and clarity inside one compact lab instead of breadth that would be harder to finish well.

One thing I liked about the final result is that the experiments are easy to repeat in front of someone else. That was important for this course because a project like this should be understandable without a long setup explanation. A person can sign in, click a preset, create or edit a task, and immediately see how the behavior changes. The automation backs that up, but the interactive workflow is what makes the project feel convincing in a short presentation.

4. Conclusion

This project ended up being a practical middle ground between a normal CRUD demo and an over-ambitious research framework. *TimeBox Testing Lab* shows that a small web application can still be a useful test-flakiness teaching tool if the unstable behaviors are chosen carefully and exposed clearly. The final version supports reproducible bug profiles, session-safe fault toggling, guided demo presets, editable tasks, passing UI/API automation, and a cleaner visual presentation than the early build.

The biggest lesson from the work was that test projects are not only about writing assertions. They are also about designing the surrounding experience so failures are easy to trigger, easy to isolate, and easy to explain. A global fault state, awkward API-only controls, jumpy page behavior, or poor screenshots can all make a testing project feel weaker than it really is. Fixing those issues made the project feel much more complete.

If I had more time, the next step would be adding lightweight logging or scenario history so the app could show which preset ran, when it ran, and what visible outcome followed. Even without that, the current project meets its main goal: it is

a small, understandable, and honestly pretty usable testing lab that demonstrates how fragile behavior can show up once software is pushed out of its “everything is normal” path.